The downloaded kernel images can also be extracted in Parallel:

```
$ find . -name \*.tar.xz | parallel tar xvf
```

A 'for' loop in a Bash script can be parallelised. In the following script, the file sizes of all the text files are printed:

```
#!/bin/sh

for file in `ls *.txt`; do
  ls -lh "$file"
done | cut -d' ' -f 5
```

The parallelised version is as follows:

```
$ ls *.txt | parallel "ls -lh {}" | cut -d' ' -f 5
```

The number of CPUs and cores in your system can be listed with GNU Parallel:

```
$ parallel --number-of-cpus
1
$ parallel --number-of-cores
4
```

The '-j' option specifies the number of jobs to be run in parallel. If the value 0 is given, GNU Parallel will try to start as many jobs as possible. The '+ N' option with '-j' adds N jobs to the CPU cores. For example:

```
$ find . -type f -print | parallel -j+2 ls -l {}
```

The input to GNU Parallel can also be provided in a tabular format. Suppose you want to run ping tests for different machines, you can have a text file with the first column indicating the ping count, and the second column listing the hostname or the IP address. For example:

```
$ cat hosts.txt
1 127.0.0.1
2 localhost
```

You can run the tests in parallel using the following code:

```
$ parallel -a hosts.txt --colsep ' ' ping -c {1} {2}

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.074 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.074/0.074/0.074/0.000 ms

PING localhost.localdomain (127.0.0.1) 56(84) bytes of data.
64 bytes from localhost.localdomain (127.0.0.1): icmp_seq=1
ttl=64 time=0.035 ms
64 bytes from localhost.localdomain (127.0.0.1): icmp_seq=2
ttl=64 time=0.065 ms

--- localhost.localdomain ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time
1000ms
rtt min/avg/max/mdev = 0.035/0.050/0.065/0.015 ms
```

GNU Parallel can also execute jobs on remote machines, for which you need to first test that ssh works:

```
$ SERVER1=localhost
$ ssh $SERVER1 echo "Eureka"
guest@localhost's password:
Eureka
```

You can then invoke commands or scripts to be run on SERVER1, as shown below:

```
$ parallel -S $SERVER1 echo "Eureka from " ::: $SERVER1
guest@localhost's password:
Eureka from localhost
```

Files can also be transferred to remote machines using the '-transfer' option. Rsync (see 'References') is used internally for the transfer. An example is shown below:

```
$ parallel -S $SERVER1 --transfer cat ::: /tmp/host.txt
guest@localhost's password:
1 127.0.0.1
2 localhost
```

Refer to the GNU Parallel tutorial and manual page (see 'References') for more options and examples. END

References

[1] GNU Parallel. *http://www.gnu.org/software/parallel/*
[2] Rsync. *http://rsync.samba.org/*
[3] GNU Parallel tutorial. *http://www.gnu.org/software/parallel/parallel_tutorial.html*
[4] GNU Parallel manual page. *http://www.gnu.org/software/parallel/man.html*

By: Shakthi Kannan

The author is a free software developer at the Fedora project, and is also a blogger. He co-maintains the Fedora Electronic Lab project.